TELSTRA ENTERPRISE ECOSYSTEM

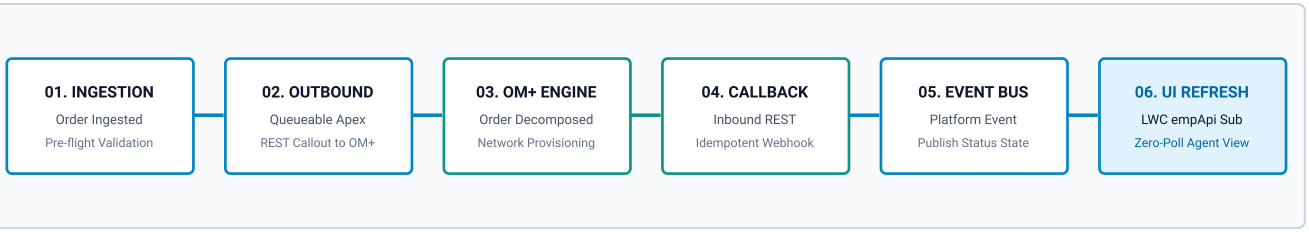
Service Cloud • OM+ Architecture

TELECOM ORDER MANAGEMENT, EVENT-DRIVEN ORCHESTRATION & AGENTFORCE AI

REST / Queueable Apex | Platform Events | LWC empApi

PLATFORM BOUNDARIES & END-TO-END ORCHESTRATION FLOW

Salesforce Service Cloud acts as the operational visibility layer for customer support, whilst **OM+ (Order Manager Plus)** handles telecom order decomposition, catalog resolution, and downstream network provisioning across Mobile, Broadband, Fixed Line, and Energy services.



CORE ENGINEERING PILLARS & FAILURE MODES

1. Async Integration & Resiliency

OM+ REST API

Synchronous callouts on order triggers risk breaching the 120s timeout and concurrent transaction caps. Built an isolated asynchronous integration pipeline.

- **Queueable Apex:** Chained execution isolates transactional boundaries and prevents locking parent Order rows.
- **Fault Tolerance:** Exponential backoff retry engine for 5xx network timeouts; persistent 4xx client errors logged to central error framework.

2. Event-Driven UI Architecture

Platform Events

Eliminated browser-side interval polling that degraded Service Console performance across thousands of concurrent telecom agents.

- **empApi Integration:** Custom Service Console LWCs subscribe to Order Status Platform Events directly on the client.
- **Live Tab Refresh:** Provisions real-time status, backend error codes, and retry actions without manual page reload.

3. ITAM Incident Correlation

Service Cloud Triage

Network activation and SIM failures in ITAM downstream systems automatically synchronize with customer Service Cases via external correlation keys.

- **Automated Triage:** Incident-driven case prioritization dynamically routes outages to specialized engineering queues.
- **SLA Protection:** Escalation Rules automatically halt or adjust SLA clocks when major network incidents are recognized.

4. Agentforce AI Operational Support

Salesforce AI

Integrated contextual AI operational guidance directly into support agent workspaces to eliminate manual root-cause investigation across multiple systems.

- **Contextual Summarization:** Synthesizes order journey, previous provisioning attempts, and ITAM error logs in one card.
- **Guided Next-Best Actions:** Recommends specific remediation paths, decreasing Average Handling Time (AHT) significantly.

APEX ENTERPRISE AUTOMATION

TRIGGER-HANDLER FRAMEWORKS, BULKIFICATION & GOVERNOR SAFE PATTERNS

Apex Architecture

1. Auto-Generate Service Orders (Trigger + Handler Pattern)

Order • After Insert, After Update

Business Problem: When customer orders are created or updated with product bundles (e.g. Triple-Play: Internet + TV + Phone), multiple downstream child service orders must be generated automatically for distinct provisioning paths.

```
trigger ServiceOrderCreationTrigger on Order (after insert, after update) { List<Order> targetOrders = new List<Order>(); for (Order ord : Trigger.new) { if (Trigger.isInsert && ord.Product_Bundle__c != null) { targetOrders.add(ord); } if (Trigger.isUpdate) { Order oldOrder = Trigger.oldMap.get(ord.Id); Boolean bundleChanged = ord.Product_Bundle__c != oldOrder.Product_Bundle__c; if (bundleChanged && ord.Product_Bundle__c != null) { targetOrders.add(ord); } } } if (!targetOrders.isEmpty()) { ServiceOrderCreationHandler.createServiceOrders(targetOrders); } }
```

Execution Pipeline:

1. **Target Filtering:** Isolates only newly created bundles or updated records where the bundle pointer mutated.
2. **Catalog Lookup:** Handler queries Product Catalog to resolve active services tied to the bundle (Fiber, VoIP, IPTV).
3. **Child Record Creation:** Instantiates one `Service_Order__c` record per individual service with default SLAs.
4. **Bulk Safe DML:** Populates Parent Order ID link and commits all child records via a single transactional DML.

ARCHITECTURAL PATTERN COMPARISON

Automation Pattern	Execution Phase	Governor Guardrail	Primary Business Value
Bundle Decomposition	After Insert / Update	Single DML statement commit; SOQL outside loops	Zero manual service ticket creation; instant order routing
Technician Auto-Assign	Before Insert / Update	Zero DML operations (direct in-memory field mutation)	Prevents dispatch delays; automated operational triage
OM+ Orchestration	Queueable Apex	Decoupled 120s HTTP timeout; async concurrency cap	End-to-end multi-cloud provisioning resilience

2. Auto-Assign Technician (Before-Save In-Memory Optimization)

Service_Order__c • Before Insert, Before Update

Business Problem: Field technician assignments must occur instantly based on customer city and service type without generating wasteful DML overhead or recursive trigger executions.

```
trigger AutoAssignTechnicianTrigger on Service_Order__c (before insert, before update) { List<Service_Order__c> toAssign = new List<Service_Order__c>(); for (Service_Order__c so : Trigger.new) { if (Trigger.isInsert) { toAssign.add(so); } if (Trigger.isUpdate) { Service_Order__c oldSO = Trigger.oldMap.get(so.Id); if (so.Customer_City__c != oldSO.Customer_City__c || so.Product_Type__c != oldSO.Product_Type__c) { toAssign.add(so); } } } if (!toAssign.isEmpty()) { TechnicianAssignmentHandler.assignTechnicians(toAssign); } }
```

Handler Execution Highlights:

1. **City Set Compilation:** Collects target cities from incoming records into a Set to avoid redundant queries.
2. **Map-Based Bulkification:** SOQL queries available resources into `Map<String, List<Technician__c>>` keyed by City.
3. **In-Memory Assignment:** Assigns first available resource directly to the trigger reference (`Trigger.new`).
4. **Exception Handling:** Automatically updates status to **"No Technician Available"** if no matching active resource is found.

FLOW ARCHITECTURE PART I

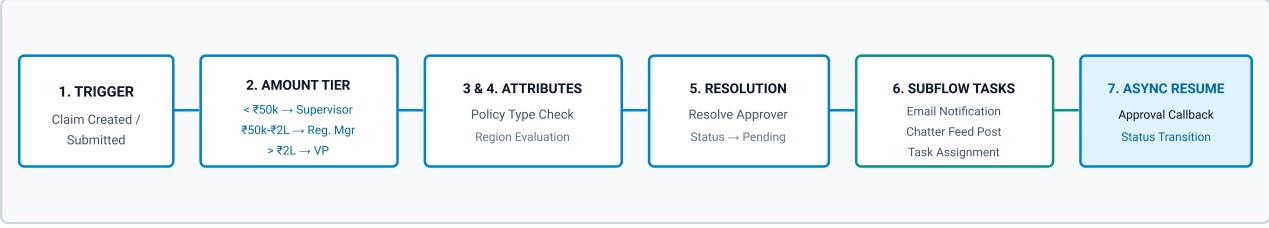
MULTI-LEVEL APPROVALS & HIGH-SPEED BEFORE-SAVE VALIDATION

Insurance Automation

FLOW #1: Multi-Level Claim Approval Flow

Record-Triggered Flow (After Save) • Claim__c

Purpose: Automate multi-factor claim routing dynamically without hardcoded approval IDs, integrating multi-channel notifications and asynchronous resume patterns.



Architectural Strengths:

- Dynamic assignment matrices eliminate hardcoded User/Group IDs.
- Subflows isolate transactional chatter and notification steps.

Enterprise Value:

- Zero manual routing overhead across complex regional matrices.
- Auditable approval histories maintained with asynchronous resume.

FLOW #4: High-Performance Claim Validation Flow

Before-Save Flow • Claim__c

Purpose: Enforce critical business rules and compute financial estimates immediately during record creation, blocking invalid claims prior to database commit.

Validation Decision Nodes:

1. **Coverage Breach:** If `Claim_Amount__c > Policy_Coverage__c` → Flag Status as **"Invalid"**.
2. **Policy Expiration:** If Policy is expired → Update Status to **"Policy Expired – Claim Not Permitted"**.
3. **Document Audit:** If required documentation is absent → Update `Submit_Status__c = "Incomplete"`.

Pre-Save Computations & Impact:

- A **Auto-Calculations:** Computes Deductible and Expected Payout in memory without issuing DML updates.
- B **Gating:** Prevents invalid records from triggering downstream Approval Workflows.
- C **Business Impact: 50% reduction** in manual adjuster audit triage time; 10x faster execution than Apex triggers.

Architectural Takeaway: Before-Save vs. After-Save Flow Selection

Before-Save flows execute up to 10× faster than After-Save triggers by operating directly on the incoming memory buffer without consuming DML statements or firing secondary trigger recursion loops. Use Before-Save for field normalization and validations; use After-Save for cross-object record creation and external events.

FLOW ARCHITECTURE PART II

SCHEDULED BATCHES, INTERACTIVE SCREEN AUTOMATION & LOAD BALANCING

Operations Dossier

FLOW #2: Renewal Reminder

Scheduled 2:00 AM

Object: Policy__c

1. Runs daily at 2:00 AM batch window.
2. Queries records where `Expiration_Date__c = TODAY() + 30`.
3. Loops collection → Creates email notification.
4. Sets `Renewal_Reminder_Sent__c = TRUE`.
5. Generates Agent Task (Due in 5 days).
6. (Optional) Dispatches SMS via subflow.

Value: Proactive SLA tracking; prevents lost revenue from lapsed renewals.

FLOW #3: Doc Generation

Screen + RT Flow

Target: Claim Adjusters

1. Screen Flow captures Claim Type, Damage, and required docs.
2. Queries active template records dynamically.
3. Auto-creates linked `Document__c` rows.
4. Fires secondary Record-Triggered Flow to dispatch checklist via Email-to-Case.

Value: 60–70% reduction in manual adjuster drafting time; standardized audit trail.

FLOW #5: Agent Load Balancing

Record-Triggered

Object: Policy__c

1. Triggered on new Policy insertion.
2. Queries `Agent__c` sorted by active workload (ASC), region, and skill rating.
3. Matches agent with lowest workload.
4. Creates `Assignment__c` audit link record.
5. Dispatches agent alert notification.

Value: Eliminates queue bottlenecks; fair load distribution; full audit tracking.

COMPREHENSIVE ENGINEERING ARCHITECTURE MATRIX

System / Automation	Automation Pattern	Governor & Resiliency Guardrail	Core Business Impact
Telstra OM+ Integration	Queueable Apex + REST Callout	120s timeout isolation; exponential backoff	Scalable multi-service telecom order provisioning
Telstra Live Console UI	Platform Events + LWC empApi	Event-driven; zero client polling overhead	Real-time error visibility; reduced call AHT
Order Decomposition Trigger	After-Save Apex Trigger + Handler	Single bulk DML commit; SOQL outside loops	Auto child creation for Internet, TV, and Phone
Technician Auto-Assign Trigger	Before-Save Apex Trigger + Handler	Zero DML statements; in-memory Map lookup	Immediate city-based technician matching
Flow 1: Claim Approval	After-Save Record-Triggered Flow	Decoupled subflows; dynamic approver queries	Tiered multi-factor authorization routing
Flow 2: Renewal Reminder	Scheduled-Triggered Batch Flow	Off-peak execution; filtered SOQL predicates	30-day proactive SLA outreach; retention safeguard
Flow 3: Doc Generation	Screen Flow + Record-Triggered Flow	Template-driven record generation	60–70% drop in manual claims adjuster paperwork
Flow 4: Claim Validation	Before-Save Record-Triggered Flow	10x faster execution; zero DML footprint	50% reduction in audit triage; blocks invalid claims
Flow 5: Load Balancing	Record-Triggered Assignment Flow	Dynamic sorting by active volume & region	Eliminates agent burnout; transparent triage history

Interview Executive Summary Note

This portfolio balances enterprise Apex programming patterns (bulkification, trigger frameworks, queueable integrations, in-memory before-save optimizations) with modern declarative tools (Platform Events, before-save validations, scheduled batch flows, and Agentforce AI workflows).